Transformer#

https://pytorch.org/docs/stable/generated/torch.nn.modules.transformer.Trans

Description:

A basic transformer layer. This Transformer layer implements the original Transformer architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build an efficient transformer layer from building blocks in core or using higher level libraries from the PyTorch Ecosystem. d_model (int) – the number of expected features in the encoder/decoder inputs (default=512). nhead (int) – the number of heads in the multiheadattention models (default=8). num_encoder_layers (int) – the number of sub-encoder-layers in the encoder (default=6).

Function Signature

```
class torch.nn.modules.transformer.Transformer(d_model=512,
nhead=8, num_encoder_layers=6, num_decoder_layers=6,
dim_feedforward=2048, dropout=0.1, activation=<function
relu>, custom_encoder=None, custom_decoder=None,
layer_norm_eps=1e-05, batch_first=False, norm_first=False,
bias=True, device=None, dtype=None)[source]#

Parameters

forward(src, tgt, src_mask=None, tgt_mask=None,
memory_mask=None, src_key_padding_mask=None,
tgt_key_padding_mask=None, memory_key_padding_mask=None,
src_is_causal=None, tgt_is_causal=None,
memory_is_causal=False)[source]#

Parameters

Return type
```

Returns:

Tensor

Code Examples

```
>>> transformer_model = nn.Transformer(nhead=16,
num_encoder_layers=12)
>>> src = torch.rand((10, 32, 512))
>>> tgt = torch.rand((20, 32, 512))
>>> out = transformer_model(src, tgt)
```

```
>>> output = transformer_model(
...     src, tgt, src_mask=src_mask, tgt_mask=tgt_mask
... )
```

Notes & Warnings

NOTE

Note If a boolean tensor is provided for any of the [src/tgt/memory]_mask arguments, positions with a True value are not allowed to participate in the attention, which is the opposite of the definition for attn_mask in torch.nn.functional.scaled_dot_product_attention().

Detailed Documentation

Documentation

A basic transformer layer.

This Transformer layer implements the original Transformer architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build an efficient transformer layer from building blocks in core or using higher level libraries from the PyTorch Ecosystem.

d_model (int) – the number of expected features in the encoder/decoder inputs (default=512).

nhead (int) – the number of heads in the multiheadattention models (default=8).

num_encoder_layers (int) – the number of sub-encoder-layers in the encoder (default=6).

num_decoder_layers (int) – the number of sub-decoder-layers in the decoder (default=6).

dim_feedforward (int) – the dimension of the feedforward network model (default=2048).

dropout (float) – the dropout value (default=0.1).

activation (Union[str, Callable[[Tensor], Tensor]]) – the activation function of encoder/decoder intermediate layer, can be a string (“relu” or “gelu”) or a unary callable. Default: relu

custom_encoder (Optional[Any]) – custom encoder (default=None).

custom_decoder (Optional[Any]) – custom decoder (default=None).

layer_norm_eps (float) – the eps value in layer normalization components (default=1e-5).

batch_first (bool) – If True, then the input and output tensors are provided as (batch, seq, feature). Default: False (seq, batch, feature).

norm_first (bool) – if True, encoder and decoder layers will perform LayerNorms before other attention and feedforward operations, otherwise after. Default: False (after).

bias (bool) – If set to False, Linear and LayerNorm layers will not learn an additive bias. Default: True.

Note: A full example to apply nn.Transformer module for the word language model is available in pytorch/examples

Take in and process masked source/target sequences.

If a boolean tensor is provided for any of the [src/tgt/memory]_mask arguments, positions with a True value are not allowed to participate in the attention, which is the opposite of the definition for attn_mask in torch.nn.functional.scaled_dot_product_attention().

src (Tensor) – the sequence to the encoder (required).

tgt (Tensor) – the sequence to the decoder (required).

src_mask (Optional[Tensor]) – the additive mask for the src sequence (optional).

tgt_mask (Optional[Tensor]) – the additive mask for the tgt sequence (optional).

memory_mask (Optional[Tensor]) – the additive mask for the encoder output (optional).

src_key_padding_mask (Optional[Tensor]) – the Tensor mask for src keys per batch (optional).

tgt_key_padding_mask (Optional[Tensor]) – the Tensor mask for tgt keys per batch (optional).

memory_key_padding_mask (Optional[Tensor]) – the Tensor mask for memory keys per batch (optional).

src_is_causal (Optional[bool]) – If specified, applies a causal mask as src_mask. Default: None; try to detect a causal mask. Warning: src_is_causal provides a hint that src_mask is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

tgt_is_causal (Optional[bool]) – If specified, applies a causal mask as tgt_mask. Default: None; try to detect a causal mask. Warning: tgt_is_causal provides a hint that tgt_mask is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

memory_is_causal (bool) – If specified, applies a causal mask as memory_mask. Default: False. Warning: memory_is_causal provides a hint that memory_mask is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

src: (S,E)(S, E)(S,E) for unbatched input, (S,N,E)(S, N, E)(S,N,E) if batch_first=False or (N, S, E) if batch_first=True.

tgt: (T,E)(T, E)(T,E) for unbatched input, (T,N,E)(T, N, E)(T,N,E) if batch_first=False or (N, T, E) if batch_first=True.

src_mask: (S,S)(S, S)(S,S) or (N·num_heads,S,S)(N\cdot\text{num_heads}, S, S) (N·num_heads,S,S).

tgt_mask: (T,T)(T, T)(T,T) or (N·num_heads,T,T)(N\cdot\text{num_heads}, T, T) (N·num_heads,T,T).

memory_mask: (T,S)(T, S)(T,S).

src_key_padding_mask: (S)(S)(S) for unbatched input otherwise (N,S)(N, S)(N,S).

tgt_key_padding_mask: (T)(T)(T) for unbatched input otherwise (N,T)(N, T)(N,T).

memory_key_padding_mask: (S)(S)(S) for unbatched input otherwise (N,S)(N, S)(N,S).

Note: [src/tgt/memory]_mask ensures that position iii is allowed to attend the unmasked positions. If a BoolTensor is provided, positions with True are not allowed to attend while False values will be unchanged. If a FloatTensor is provided, it will be added to the attention weight. [src/tgt/memory]_key_padding_mask provides specified elements in the key to be ignored by the attention. If a BoolTensor is provided, the positions with the value of True will be ignored while the position with the value of False will be unchanged.

output: (T,E)(T, E)(T,E) for unbatched input, (T,N,E)(T, N, E)(T,N,E) if batch_first=False or (N, T, E) if batch_first=True.

Note: Due to the multi-head attention architecture in the transformer model, the output sequence length of a transformer is same as the input sequence (i.e. target) length of the decoder.

where SSS is the source sequence length, TTT is the target sequence length, NNN is the batch size, EEE is the feature number

Generate a square causal mask for the sequence.

The masked positions are filled with float('-inf'). Unmasked positions are filled with float(0.0).